A Practical Method for Detecting Secrets in Environment Variables (Application Note, 2026 v1.0)

Jonas Åström

Abstract. This application note presents an implementation-oriented method for estimating secret probability in environment variables (e.g., API keys, access tokens, and strong passwords) in real screening pipelines. The method combines permutation-aware entropy diagnostics with Bayesian evidence fusion; to our knowledge, this specific combination (permutation-aware entropy baselines plus Bayesian posterior evidence updates for environment-variable screening) is not standard in widely used secret-scanning tools. The presentation includes a practical hypothesis-testing interpretation in which a permutation-derived statistic serves as the test quantity and is approximated in practice with a fixed small number of permutations, yielding an interpretable and reproducible detector.

Problem Setting

Given an environment entry of the form `NAME=VALUE`, the goal is to estimate

$$P(\text{secret} \mid \text{evidence from NAME and VALUE}).$$

The method is designed to reduce both false positives (paths, standard shell variables) and false negatives (high-entropy but structured credentials).

Throughout, S denotes a candidate value string; H_1 and H_2 denote character-level and 2-gram entropy; R_c and R_2 denote entropy ratios against permutation baselines; and $P(\text{secret})$ denotes the posterior after sequential evidence updates.

Positioning with Prior Work

This application note reports a practical implementation. Prior empirical work has documented the prevalence and impact of secret leakage in public repositories [3], and follow-up studies have shown that deployment challenges extend beyond raw false-positive rates [4]. In operational tooling, our method sits alongside established scanners such as GitHub Secret Scanning [5], Yelp's `detect-secrets` [6], and `gitleaks` [7]. Public documentation and maintainer descriptions indicate that common approaches rely on regex/rule matching, entropy thresholds, and allowlists (for example, in `gitleaks`) [8], while permutation-based entropy baselines are not explicitly described. The focus here is an explicit, reproducible combination of permutation-aware entropy features and Bayesian evidence fusion for environment-variable screening.

Table 1 gives a scope-focused comparison: many established tools target broad repository/text scanning and operational integration, whereas this note isolates scoring for `NAME=VALUE` screening and emphasizes a fused, permutation-calibrated evidence model over plain Shannon-only scoring.

Approach class	Primary scope	Indicators / statistics	Inference model
Repository secret scanners (e.g., GitHub Secret Scanning [5], `detect-secrets` [6], `gitleaks` [7,8])	Large-text/repository detection plus operational workflow concerns	Rule/pattern matching, entropy-threshold heuristics, and allowlist controls for practical detection pipelines	Rule- and heuristic-driven scoring designed for actionable detection decisions in operational workflows
This application note (environment-variable scorer)	Focused `NAME=VALUE` scoring method, separated from broader systems pipeline concerns	Deterministic permutation-normalized conditional-entropy and 2-gram entropy ratios (R_c, R_2), alongside Shannon entropy (H_1)	Interpretable Bayesian posterior updates over entropy and non-entropy cues

Table 1: Scope and method comparison between broad secret-scanning tool classes and the scoring method documented here.

Entropy Features and Test Statistics

The detector computes four entropy-based features from the value string:

1) Shannon entropy H_1 on characters. For a character distribution $p(x)$ over the string,

$$H_1 = -\sum_x p(x) \log_2 p(x).$$

2) 2-gram entropy H_2. Let $p_2(x, y)$ denote the empirical distribution of adjacent character pairs; then

$$H_2 = -\sum_{x,y} p_2(x, y) \log_2 p_2(x, y).$$

3) Relative conditional entropy ratio

$$R_c = \frac{H(X_i \mid X_{i-1})}{\operatorname{median}_\pi H\left(X_i^\pi \mid X_{i-1}^\pi\right)}.$$

Here,

$$H(X_i \mid X_{i-1}) = -\sum_{x,y} p(x, y) \log_2 p(y \mid x),$$

with $p(x, y)$ estimated from adjacent character pairs in the string. Here, π denotes a deterministic pseudo-random permutation of character order. If the denominator collapses to zero, the implementation returns a sentinel ratio of 2.0.

4) Relative 2-gram entropy ratio

$$R_2 = \frac{H_2(\text{original})}{\operatorname{median}_\pi H_2(\text{permuted})},$$

with the same deterministic permutation scheme.

Thus, permutation calibration is applied to both sequence-sensitive features: conditional 1-gram entropy (via R_c) and adjacent 2-gram entropy (via R_2). The current fusion layer then treats these as separate evidence terms and updates the posterior sequentially (one Bayes update per feature), together with normalized H_1 and later name/format cues.

For short strings (length < 12), the method falls back to direct entropy (without permutation), avoiding unstable statistics.

These ratios can be interpreted as test statistics for deviation from a permutation-based null reference.

Hypothesis-Testing Formalization (Practical Form)

Let S be a value string, and let $T(S)$ be a statistic such as R_c or R_2. Define hypotheses

$$H_0: \text{ non-secret-like generation} \quad \text{vs.} \quad H_1: \text{ secret-like generation.}$$

Using a permutation reference distribution $\{T(S^{\pi_b})\}_{b=1}^{B}$, a two-sided permutation-style p-value can be written as

$$\hat{p}_B = \frac{1 + \sum_{b=1}^{B} \mathbf{1}\{|T(S^{\pi_b}) - \mu_B| \geq |T(S) - \mu_B|\}}{B+1},$$

where μ_B is the empirical center (e.g., median) of the permuted statistics.

In the current implementation, this idealized test is approximated with a fixed, small B (typically 5 or 10) and summarized through robust ratios (median-normalized entropy statistics) rather than explicit p-values. In practice, permutation testing is used mainly as a calibration mechanism for stable feature construction, and R_c and R_2 can be treated as calibrated summary features without computing explicit p-values.

Deterministic Permutation Procedure

For each permutation index, a seeded Fisher–Yates shuffle generates a reproducible rearrangement of the string. The implementation evaluates entropy across several such permutations (typically 5 or 10) and uses the median as a robust baseline.

Empirically, secrets often preserve near-random behavior even after permutation, whereas structured non-secrets can exhibit larger deviations from the permuted baseline.

For finite B, Monte Carlo approximation error decreases as B grows. In this specific setting, each summand

$$I_b = \mathbf{1}\{|T(S^{\pi_b}) - \mu_B| \geq |T(S) - \mu_B|\}$$

is an indicator variable, so $I_b \in \{0, 1\}$ and is therefore bounded. If permutations are sampled independently (or are sufficiently decorrelated in practice), then $\{I_b\}$ behaves as Bernoulli draws with mean equal to the tail probability being estimated. Hence, standard concentration bounds (e.g., Hoeffding's inequality) apply [1,2]:

$$\Pr\left(\left|\frac{1}{B}\sum_{b=1}^{B} I_b - \mathbb{E}[I_b]\right| \geq \varepsilon\right) \leq 2\exp(-2B\varepsilon^2).$$

This yields a typical estimation error of order $O(B^{-1/2})$, clarifying the speed–accuracy tradeoff when choosing small fixed B.

Assumptions and Limitations

The method relies on several practical assumptions that should be made explicit:

- **Permutation sampling assumption:** the concentration statement for $\hat{p}_B$ is exact under independent permutation samples; deterministic seeded permutations are used here as a practical approximation.
- **Feature-likelihood calibration:** Bayesian likelihood functions are currently heuristic and should be re-calibrated when deployment domains change.
- **Short-string regime:** when length < 12, permutation features are bypassed; decisions then rely more heavily on Shannon entropy and non-entropy cues.
- **Domain drift risk:** naming conventions, token formats, and path conventions can evolve over time, requiring periodic updates to prefixes, patterns, and thresholds.

Bayesian Evidence Fusion Layer

The model uses a prior

$$P(\text{secret}) = 0.10$$

and updates it sequentially with likelihood models.

Entropy likelihoods:

- The model computes three entropy-derived evidence terms: normalized Shannon entropy H_1, permutation-calibrated conditional 1-gram ratio R_c, and permutation-calibrated 2-gram ratio R_2.
- Ratios near 1.0 are modeled as more likely under the secret hypothesis using a Gaussian-like likelihood.
- Ratios far from 1.0 increase non-secret likelihood via a bimodal distance-based function.
- Normalized Shannon entropy is mapped through sigmoids for secret and non-secret likelihoods.
- In the default implementation, these entropy terms are fused with equal structural weight by applying one sequential Bayes update per term (rather than an extra manual weighting coefficient).

Likelihood design rationale. The selected functional forms encode practical inductive bias while keeping the model interpretable:

- **Ratio terms centered at 1.0:** if a value remains statistically similar under permutation, the ratio tends to stay near 1.0; this motivates a secret-likelihood peak around 1.0.
- **Distance-based non-secret model:** larger deviations $|r - 1|$ are treated as stronger evidence of structure, motivating a non-secret likelihood that increases away from the center.
- **Sigmoid for normalized Shannon entropy:** smooth monotone mapping avoids hard cutoffs and reflects that higher normalized entropy generally supports the secret hypothesis.
- **Floors and ceilings on likelihoods:** bounded likelihood values prevent single features from numerically dominating all subsequent updates.

These parameter values are heuristic, implementation-oriented defaults; in deployment they should be re-estimated or calibrated on labeled data.

Independence and ordering caveat. Sequential Bayes updates are exact only when likelihood terms are conditionally independent given the hypothesis. In practice, entropy, name-pattern, and prefix features are partially correlated, so unless a joint model or dependency correction is introduced, the resulting posterior is best interpreted as a calibrated score rather than a strictly coherent posterior probability.

Name and format evidence mapped to likelihoods. Additional non-entropy evidence is incorporated as explicit likelihood terms:

- Known secret prefixes in values (e.g., `sk_`, `AKIA`, `ghp_`) map to high secret likelihood and low non-secret likelihood.
- Secret-like variable names (containing terms such as `KEY`, `TOKEN`, `PASSWORD`) map to higher secret likelihood.
- Path-like values (e.g., `/home/...`, `/tmp/...`) map to high non-secret likelihood and low secret likelihood.

Each evidence source is incorporated via Bayes updates:

$$P' = \frac{P\,L_s}{P\,L_s + (1-P)\,L_{\neg s}},$$

where L_s and $L_{\neg s}$ are likelihoods under secret and non-secret hypotheses.

Screening Pipeline

For each environment variable:

1. Split into `name` and `value`.
2. Skip empty values, very short values, and a whitelist of common non-secret variables.
3. Compute entropy-based posterior from H_1, R_c, and R_2.
4. Update with prefix, name-pattern, and path evidence.
5. Flag as potential secret if posterior exceeds a configurable threshold.

For a compact operational view, Figure 1 summarizes the full screening flow (including branch-specific checks and evidence fusion), Table 2 lists practical defaults used in the current implementation, and Algorithm 1 gives step-by-step pseudocode.

Concrete Walkthroughs (Two Redacted Examples)

This section instantiates the full pipeline on two redacted entries: one clear case and one flagged case.

Walkthrough A (clear case): input and thresholds.

- `name = MEMORY_PRESSURE_WRITE`
- `value = [VALUE REDACTED]` (length = 28)
- Minimum length threshold = 24
- Decision threshold $\tau = 0.50$

Step 1: parse and prechecks. The variable is parsed as non-empty, not whitelisted, and above the minimum length, so scoring proceeds.

Step 2: entropy and structure indicators. For the redacted value, the feature extractor reports:

$$H_1 = 3.3513 \text{ bits/char}, \quad \frac{H_1}{8} = 0.4189, \quad R_c = 0.6958, \quad R_2 = 0.8992.$$

Both ratio features are below 1.0, indicating sequence structure relative to permutation baselines. The format checks also report: no known secret prefix, no secret-like variable-name pattern, no path-like value, and base64-like format detected.

Step 3: Bayesian evidence updates. Starting from prior

$$P_0 = P(\text{secret}) = 0.10,$$

the entropy evidence uses the following likelihood pairs from the run output:

- $R_c = 0.6958$: $P(E \mid \text{secret}) = 0.2087$, $P(E \mid \neg\text{secret}) = 0.9499$
- $R_2 = 0.8992$: $P(E \mid \text{secret}) = 0.7781$, $P(E \mid \neg\text{secret}) = 0.1000$
- $H_1/8 = 0.4189$: $P(E \mid \text{secret}) = 0.1405$, $P(E \mid \neg\text{secret}) = 0.8595$

After sequential entropy and non-entropy updates (prefix absent, name-pattern absent, path absent), the final posterior reported by the run is:

$$P_{\text{final}} = 0.0140.$$

Step 4: threshold decision. With threshold $\tau = 0.50$,

$$P_{\text{final}} = 0.0140 < 0.50,$$

so the entry is classified as *Do not flag* (CLEAR).

This example shows how moderate entropy, sub-unit permutation ratios, and absent secret-specific contextual signals can combine into a low posterior under the current likelihood model and thresholds.

Walkthrough B (flagged case): input and thresholds.

- `name = MINIMAX_API_KEY`
- `value = [VALUE REDACTED]` (length = 125)
- Minimum length threshold = 24
- Decision threshold $\tau = 0.50$

Step 1: parse and prechecks. The variable is parsed as non-empty, not whitelisted, and above the minimum length, so scoring proceeds.

Step 2: entropy and structure indicators. For the redacted value, the feature extractor reports suffix-based Shannon entropy (after detected prefix):

$$H_1 = 5.5116 \text{ bits/char}, \quad \frac{H_1}{8} = 0.6890, \quad R_c = 1.0209, \quad R_2 = 1.0047.$$

Both ratio features are slightly above 1.0, indicating near-random behavior relative to permutation baselines. The format checks report: known secret prefix present, secret-like variable-name pattern present, no path-like value, and not base64-like.

Step 3: Bayesian evidence updates. Starting from prior

$$P_0 = P(\text{secret}) = 0.10,$$

the entropy evidence uses the following likelihood pairs from the run output:

- $R_c = 1.0209$: $P(E \mid \text{secret}) = 0.9418$, $P(E \mid \neg\text{secret}) = 0.1000$
- $R_2 = 1.0047$: $P(E \mid \text{secret}) = 0.9496$, $P(E \mid \neg\text{secret}) = 0.1000$
- $H_1/8 = 0.6890$: $P(E \mid \text{secret}) = 0.7088$, $P(E \mid \neg\text{secret}) = 0.2912$

After sequential entropy and non-entropy updates (prefix present, name-pattern present, path absent), the final posterior reported by the run is:

$$P_{\text{final}} = 1.0000.$$

Step 4: threshold decision. With threshold $\tau = 0.50$,

$$P_{\text{final}} = 1.0000 > 0.50,$$

so the entry is classified as *Flag as potential secret* (FLAGGED).

Together, these two walkthroughs illustrate both ends of the operating range under identical screening thresholds: a low-score clear case and a high-confidence flagged case.

Parameter	**Default**	**Role**
Prior secret probability $P(\text{secret})$	0.10	Baseline posterior before evidence updates
Permutation count B	5 or 10	Number of deterministic shuffles used for permutation baselines
Short-string cutoff	12 chars	Below cutoff, permutation-based entropy is skipped
Minimum value length for scoring	implementation-configured	Early prefilter to avoid unstable or non-informative scoring
Posterior flag threshold	deployment-configured	Decision boundary for "flag as potential secret"

Table 2: Practical default parameters and their operational role.

Failure Modes and Practical Checks

Key failure modes and practical mitigations include:

- **High-entropy non-secrets:** random identifiers that are not credentials may score high; mitigate with tighter name/prefix evidence.
- **Structured secrets:** secrets with strong formatting constraints may have lower entropy ratios; mitigate with prefix and name cues.
- **Path/value ambiguity:** uncommon path-like strings can be misclassified; mitigate by adjusting path heuristics for local environments.
- **Threshold drift:** operating point may shift across systems; mitigate by periodic threshold tuning on labeled samples.

Practical Characteristics

- **Interpretable:** each contribution is an explicit Bayesian update.

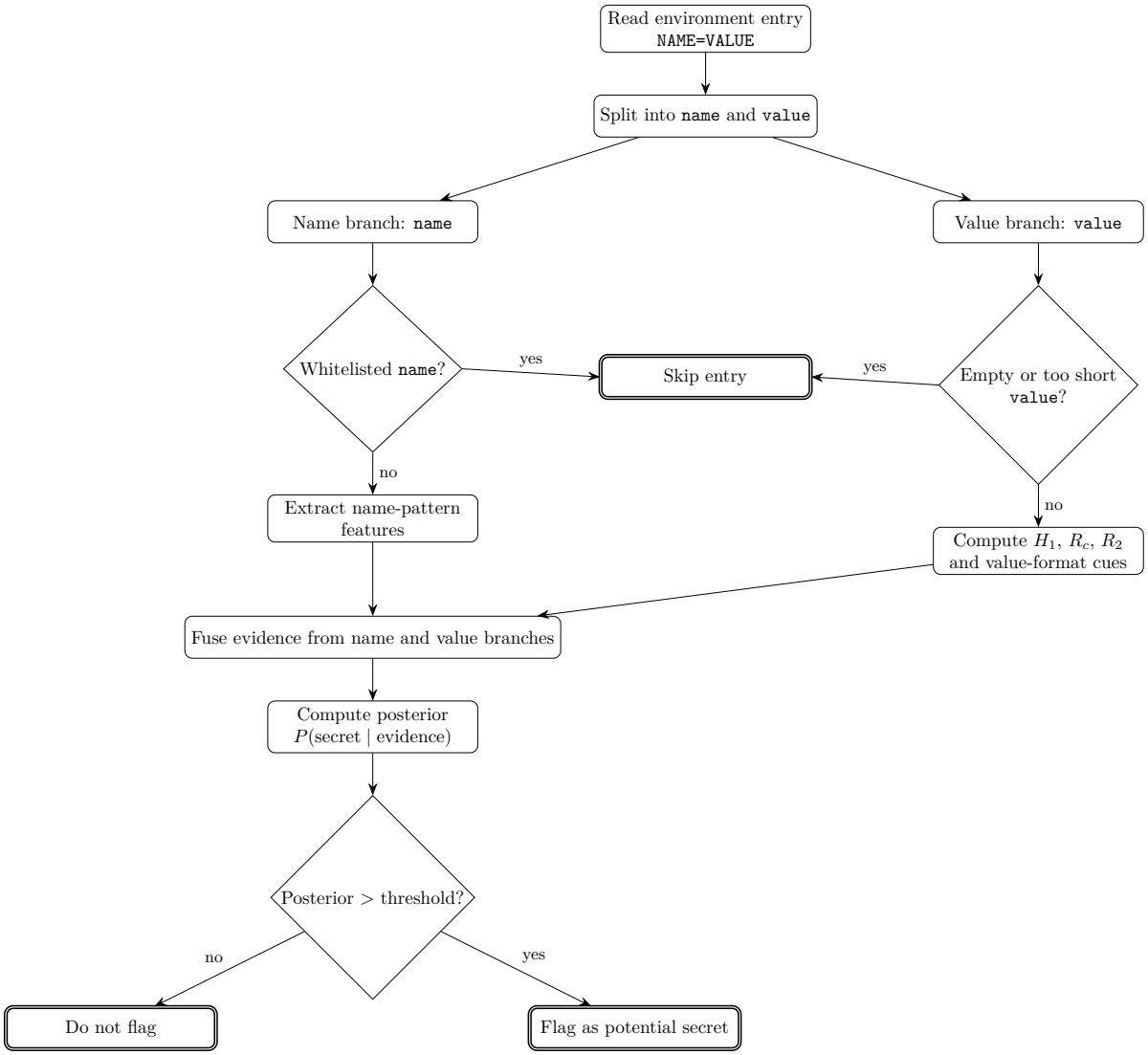

Figure 1: Screening pipeline flow graph.

- **Deterministic:** permutation seeds are fixed by index, enabling reproducible scans.
- **Robust:** median over permutations reduces sensitivity to outlier shuffles.
- **Configurable:** threshold and minimum length can be tuned for precision/recall tradeoffs.

Summary

This application note presents an implementation-oriented method for estimating secret probability in environment variables by combining permutation-aware entropy diagnostics with Bayesian evidence fusion over string statistics, naming cues, known prefixes, and path indicators. It further provides a practical hypothesis-testing interpretation, explicit assumptions and limitations, and reproducible screening defaults for operational pipelines.

Algorithm 1 Operational screening pseudocode

Require: Environment entry `NAME=VALUE`, posterior threshold τ
Ensure: One of *Skip entry*, *Do not flag*, *Flag as potential secret*

Parse entry into `name` and `value`
if `name` is whitelisted **then**
    return *Skip entry*
end if
if `value` is empty or too short **then**
    return *Skip entry*
end if
Compute entropy features with short-string fallback:

$$H_1 = -\sum_i p_i \log_2 p_i,\ R_c = \frac{H(X_i \mid X_{i-1})}{\operatorname{median}_\pi H(X_i^\pi \mid X_{i-1}^\pi)},\ R_2 = \frac{H_2(\text{original})}{\operatorname{median}_\pi H_2(\text{permuted})}$$

Initialize posterior $P \leftarrow P(\text{secret}) = 0.10$
Compute entropy likelihood pairs:
For each $r \in \{R_c, R_2\}$, let $d = |r - 1|$ and set

$$L_s^r = 0.1 + 0.85 \exp\left(-\tfrac{1}{2}\left(\tfrac{r-1}{0.15}\right)^2\right),$$

$$L_{\neg s}^r = \begin{cases} 0.1, & d < 0.3 \\ 0.5 + 0.45 \exp\left(-\tfrac{1}{2}\left(\tfrac{d-0.3}{0.2}\right)^2\right), & d \geq 0.3 \end{cases};$$

normalize Shannon entropy as $h = H_1/8$ and set

$$L_s^{H_1} = \frac{1}{1 + \exp(-10(h - 0.6))},\ L_{\neg s}^{H_1} = \frac{1}{1 + \exp(-10(0.6 - h))}$$

For $e \in \{H_1, R_c, R_2\}$, update posterior with

$$P \leftarrow \frac{P\, L_s^e}{P\, L_s^e + (1 - P)\, L_{\neg s}^e}$$

Compute additional evidence: known prefix, name pattern, path-like indicator
Update posterior with additional evidence using

$$P \leftarrow \frac{P\, L_s}{P\, L_s + (1 - P)\, L_{\neg s}}$$

if posterior $> \tau$ **then**
    return *Flag as potential secret*
else
    return *Do not flag*
end if

References

[1] W. Hoeffding, "Probability Inequalities for Sums of Bounded Random Variables," *Journal of the American Statistical Association*, vol. 58, no. 301, pp. 13–30, 1963.

[2] S. Boucheron, G. Lugosi, and P. Massart, *Concentration Inequalities: A Nonasymptotic Theory of Independence*. Oxford University Press, 2013.

[3] K. Meli, M. McNiece, and B. Reaves, "How Bad Can It Git? Characterizing Secret Leakage in Public GitHub Repositories," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2019.

[4] M. M. Rahman, C. Parnin, and L. Williams, "Why Secret Detection Tools Are Not Enough: It's Not Just About False Positives," *Empirical Software Engineering*, vol. 27, no. 3, 2022.

[5] GitHub Docs, "About Secret Scanning," Accessed 2026. [Online]. Available: `https://docs.github.com/code-security/s`

[6] Yelp, "detect-secrets," Accessed 2026. [Online]. Available: `https://github.com/Yelp/detect-secrets`

[**7**] Gitleaks Authors, "gitleaks," Accessed 2026. [Online]. Available: `https://github.com/gitleaks/gitleaks`

[**8**] Z. Rice, "Regex is (almost) all you need: How Gitleaks combines regex, entropy, and allowlists to find secrets," *Looking at Computer*, Jan. 27, 2025.